



**universidad
veracruzana**

REPORTE DE PRACTIA 3

Realizado por:

Gil Martínez Daniel Alberto

Martínez Morales Antonio

Barradas Solís Nadia Fernanda

Muños Camacho Josué

González Avendaño Ivon Alejandra

Acosta López Eduardo

02/05/2025

Introducción

En esta práctica se implementó, utilizando un **Raspberry Pi Pico** y Python en Spyder, un sistema de monitoreo que integra cuatro variables físicas mediante sensores conectados al microcontrolador. Los datos capturados se visualizan en tiempo real en la terminal de Spyder y se almacenan de manera estructurada en un archivo CSV para su posterior análisis.

El objetivo principal fue desarrollar un flujo de datos robusto y confiable, basado en estructuras de datos eficientes, que permitiera la adquisición, transmisión y procesamiento de las mediciones en Python. A diferencia de implementaciones previas con Arduino, en este caso se aprovecharon las capacidades del Raspberry Pi Pico, como su bajo consumo energético, su interfaz flexible con sensores y su programación directa en MicroPython, lo que facilitó una integración más eficiente con el entorno de Spyder.

Los sensores utilizados fueron los siguientes:

- Sensor de temperatura y humedad
- Sensor de humo
- Sensor ultrasónico
- Sensor de proximidad

Esta implementación no solo permitió evaluar el desempeño del Raspberry Pi Pico en aplicaciones de adquisición de datos, sino también comparar su rendimiento y facilidad de uso frente a otras plataformas como Arduino.

Desarrollo

Antes de integrar todos los sensores en un solo sistema, se realizaron pruebas individuales en MicroPython para cada módulo, asegurando su correcto funcionamiento y calibración:

- Sensor DHT11
 - Se utilizó la librería dht de MicroPython para leer datos digitales.
 - Se verificó la estabilidad de las mediciones con un intervalo de muestreo de 2 segundos.
- Sensor MQ-2 (Humo)
 - Se conectó a un pin ADC (Analog-to-Digital Converter) del Pico.
- Sensor HC-SR04 (Ultrasónico)
 - Se programó la medición de distancia mediante pulsos de trigger y echo.
- Sensor de Proximidad
 - Se leyó su salida digital (HIGH/LOW) para detectar presencia.

Integración en un Solo Código.

Una vez validados los sensores de manera individual, se desarrolló un programa unificado en MicroPython que:

1. Inicializa todos los sensores y configura los pines correspondientes (digitales y analógicos).
2. Implementa un bucle principal que:
 - Lee cada sensor en secuencia.
 - Aplica filtros y ajustes según las necesidades de cada uno.
 - Formatea los datos en un diccionario de Python para facilitar su procesamiento.

Comparativa

Aspecto	Arduino	Raspberry Pi Pico	¿Se usó el mismo código en PC?	Modificaciones necesarias
Lenguaje de programación	C++ (con librerías específicas de Arduino)	MicroPython (similar a Python 3)	✗ No (Spyder usa Python estándar)	El Pico requiere código en MicroPython, mientras que la PC usa Python normal.
Comunicación Serial	<code>Serial.begin(9600) + Serial.print()</code>	<code>UART(0, baudrate=9600) + print()</code>	✓ Sí (ambos usan UART)	Mismo protocolo, pero la sintaxis de inicialización cambia.
Lectura de sensores digitales	<code>digitalRead(pin)</code>	<code>machine.Pin(pin, machine.Pin.IN)</code>	✗ No	MicroPython usa clases como <code>machine.Pin</code> .
Lectura de sensores analógicos	<code>analogRead(pin)</code>	<code>machine.ADC(pin).read_u16()</code>	✗ No	El Pico devuelve valores de 16 bits (0-65535), Arduino de 10 bits (0-1023).
Manejo de librerías	Librerías específicas (.h, .cpp)	Módulos de MicroPython (ej: <code>dht</code> , <code>utime</code>)	✗ No	Algunos sensores (como el DHT11) requieren librerías diferentes.
Temporización	<code>delay()</code> , <code>millis()</code>	<code>utime.sleep()</code> , <code>utime.ticks_ms()</code>	✗ No	MicroPython no tiene <code>millis()</code> , se usa <code>ticks_ms()</code> .
Estructura de datos	Arrays estáticos, structs	Listas, diccionarios (como Python)	✓ Sí (en PC se usa Python normal)	El Pico maneja datos como JSON, igual que la PC.
Manejo de errores	Básico (sin excepciones)	Try/Except (como Python)	✓ Sí	MicroPython permite manejo de errores más robusto.
Almacenamiento	Requiere SD externa o EEPROM	Memoria interna (o sistema de archivos)	✗ No	El Pico puede guardar datos en su flash (limitado).

Comparativa

En este trabajo de primera mano podemos denotar la simplicidad con la que se pueden obtener los datos, el solo utilizar un programa para poder interactuar con los sensores hace al Raspberry pico un microcontrolador muy útil y eficiente para nosotros como instrumentistas en la labor de del procesamiento de datos. De entre las primeras diferencias que podemos notar esta el uso de "import machine" que dicho por algunas IA es el que nos permite interactuar con el hardware de bajo nivel de la placa a diferencia de arduino que no lo necesita ya que al trabajar con C/C++ las instrucciones son un poco mas simples; esto esta ligado a la utilización única de un programa ya que otras de sus diferencias esta en el lenguaje de programación el de el pico es micropython. Una diferencia menor en cuanto a programación es la manera en la que nos referimos a los pines muy parecida algunos comandos de arduino pero en python "an1.read_u16" Sin duda el microcontrolador Raspberry pi es una muy buena opcion para multiples proyectos.

Conclusión

En esta práctica se implementó un sistema de monitoreo de variables ambientales utilizando un Raspberry Pi Pico con MicroPython, logrando una integración eficiente entre la captura de datos, su transmisión serial y el procesamiento en Python (Spyder). A diferencia de una implementación con **Arduino (C++)**, el Pico demostró ventajas clave, como su compatibilidad nativa con Python, manejo simplificado de periféricos y capacidad para estructurar datos en formatos como JSON, facilitando la comunicación con la PC.

Hallazgos principales:

- MicroPython permitió un desarrollo más ágil gracias a su sintaxis similar a Python, manejo de diccionarios y excepciones.
- Arduino requirió código en C++ con librerías específicas, limitando su flexibilidad para integración directa con Python.
- El script en Spyder (Python) funcionó sin modificaciones para ambos casos, ya que solo depende del protocolo serial, no del firmware del microcontrolador.
- El Pico demostró ser una alternativa viable a Arduino para prototipos que requieren rápida integración con herramientas Python.

En síntesis, esta práctica no solo permitió comparar dos plataformas de hardware, sino también demostrar que la elección del microcontrolador debe basarse en el contexto del proyecto: mientras Arduino ofrece precisión y control avanzado, el Pico brinda simplicidad y compatibilidad con el ecosistema Python.